

event. The *AddAction* function takes two parameters—the event, and the action. When specifying the event type, in most cases, creating a new event object inline is enough and recommended. QControl supports all events allowed by JavaScript, and you can choose any one of them. The second parameter is the action, which can be one of these three types:

1. *QServerAction*: This will reload the entire page with the changes defined in the event-handler function. When a *QServerAction* is executed, the client sends an HTTP POST request over the wire. The server receives the request and runs the event-handler function, and makes any required visual changes to the page accordingly, and then sends back the entire page, thus reloading the page.
2. *QAjaxAction*: This will render the changes without reloading the page (using AJAX). In this case, the client submits the form using an AJAX request. The server runs the event-handler function, and records any changes needed for the layout. These changes are then transformed and sent to the client for reflecting the changes on screen.
3. *QJavaScriptAction*: This will call a JavaScript function on the client side. In this case, the event-handler function must be present on the client side. No calls are made to the server.

All these actions would take one parameter—the name of the function to be called when the event occurs. In our case, the function *btnSubmit_Click* will be called when *btnSubmit* is clicked. In case of *QJavaScriptAction*, the function passed in as the argument must have the braces included in the call, and the function must be present on the client side—either included in another JS file, or inline in the document. QControl cannot validate the availability of the event-handler functions, and it is the developer's job to make sure that they exist.

Our event-handler function *btnSubmit_Click* takes in three parameters:

1. *strFormID*: This is the *FormID* (same as the class name).
2. *strControlID*: This is the control ID of the HTML element that initiated the action. Control IDs are generated automatically by QControl when a new control is created. These are the same ones used for the *id* attribute of the element when it is rendered. The developer can enforce a desired ID by supplying it as the second parameter to the constructor, like:

```
$this->lblResult = new QLabel($this, 'resultLabel');
```

This will set the control ID of *lblResult* to '*resultLabel*'. QControl allows only alphanumeric control IDs.

3. *strParameter*: This is any extra parameter that needs to be passed to the function. This parameter can be set by

using the *ActionParameter* property of the *QControl*.

In most cases, you would not need to use them, but they are needed to be present in the event-handler function. With time, you would discover that these three parameters can help you reduce coding efforts and cut short development time even more.

Now we take a look at what the event handler does. Inside the event handler, we create a new blog post object named *\$objNewPost*. (Remember, QControl creates classes for each of your tables, and your rows are treated as objects of the table class.) Like we did in the last article, we initialise the object with database defaults, set the value of the post title and body, and save the post. We also set the text of *lblResult* to 'Post was saved' and empty the text-boxes *txtTitle* and *txtBody*. There are a few points worth noting:

1. We retrieve the values input into *QTextBox* controls by just fetching their *Text* property value. The same property is used to set the values as well (as we did for the *QButton* and *QLabels*). You do not have to make any extra effort to retrieve values from *\$_POST* or *\$_GET* arrays, anywhere.
2. Though all the actions are taken via AJAX, QControl does not require us to deal with the low-level JavaScript, and *XmlHttpRequests* and responses. You do not write any HTML to be sent. Just set the values to what you want, and QControl will calculate what needs to be done by itself.
3. The event handler does not return any value—event-handler functions will not return values. If you want to terminate the function conditionally (e.g., if the input in the *txtTitle* text-box was blank), then simply return without any value.
4. The server keeps a copy of the state of the application on the client side, and every time the page updates itself via a *QServerAction* or *QAjaxAction*, the server updates and syncs the copy of the page state. You do not need to worry about what data the variables contain on the server side—they always contain the last submitted value. If you use the back button, the server knows that you clicked the back button (well, not exactly, but it's safe to assume it at the outset of our journey with QControl) and uses the corresponding state to produce the HTML. Whatever be the case, in the end, you will not have to go through anomalies.

All the code in the controller file is useless if you do not *Run* the file. To run the file, we need to call the public static function *Run*, with the class name as the parameter. This function is built into QForm, and is not meant to be written manually. It tells QControl to search for the template file in the same directory. By default, it would look for a file with the same name but ending in